



**PRESIDENCY UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013  
Italgapura, Rajankunte, Yelahanka, Bengaluru - 560064



# **Image Based Breed Recognition and Disease Detection for Cattle and Buffalo**

**A PROJECT REPORT**

**Submitted by**

**Shreyas S - 20221CSE0066**

**Sakshi Kumari - 20221CSE0388**

**Under the guidance of,**

**Dr Md Zia Ur Rahman**

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**

**PRESIDENCY UNIVERSITY**

**BENGALURU**

**MAY 2026**



# PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013  
Italgapura, Rajankunte, Yelahanka Bengaluru - 560064



## BONAFIDE CERTIFICATE

Certified that this report "Image Based Breed Recognition and Disease Detection for Cattle and Buffalo" is a bonafide work of "Shreyas S (20221CSE0066), Sakshi Kumari (20221CSE0388)", who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of B.Tech in Computer Science and Engineering during 2025-26.

  
24/04/2026

Dr. Md Zia Ur Rahman  
Project Guide  
PSCS  
Presidency University

  
24/04/26

Mr. Muthuraju V  
Program Project Coordinator  
PSCS  
Presidency University



Dr. Sampath A K  
School Project Coordinators  
PSCS  
Presidency University



Dr. Blessed Prince  
Head of the Department PSCS  
Presidency University

  
N. 28

Dr. N. Duraipandian  
Dean  
PSCS  
Presidency University

Name and Signature of the Examiners:

| Sl. No. | Name           | Signature                                                                           | Date    |
|---------|----------------|-------------------------------------------------------------------------------------|---------|
| 1       | Muthuraju      |  | 24/4/26 |
| 2       | Pravin Pandian |  | 24/4/26 |



## **Declaration**

We the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named **Shreyas S**, and **Sakshi Kumari**, hereby declare that the project work titled “**Image Based Breed Recognition and Disease Detection for Cattle and Buffalo**” has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Place: Bengaluru

Date: 23-April-2026

**Shreyas S (20221CSE0066)**

Shreyas

**Sakshi Kumari (20221CSE0388)**

Sakshi Kumari

## Acknowledgement

For completing this project work, we have received the support and the guidance from many people whom we would like to mention with a deep sense of gratitude and indebtedness. We extend our gratitude to our beloved Chancellor, Pro-Vice Chancellor, and Registrar for their support and encouragement in the completion of the project.

We would like to sincerely thank our internal guide **Dr Md Zia Ur Rahman**, Assistant Professor, Presidency School of Computer Science and Engineering, Presidency University, for his moral support, motivation, timely guidance, and encouragement provided to us during the period of our project work.

We are also thankful to **Dr. Blessed Prince**, Professor, Head of the Department, Presidency School of Computer Science and Engineering, Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, Presidency School of Computer Science and Engineering, and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of our project work.

We are grateful to **Dr. Sampath A K**, PSCS Project Coordinator, and **Mr. Muthuraju V**, Program Project Coordinator, Presidency School of Computer Science and Engineering, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to the Teaching and Non-Teaching staff of the Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

**SAKSHI KUMARI**

**SHREYAS S**



## Abstract

Agriculture and livestock management form the backbone of the economy in many developing nations. However, the manual identification of cattle breeds and the timely detection of bovine diseases like Lumpy Skin Disease (LSD) and Foot-and-Mouth Disease (FMD) present significant challenges to farmers and veterinarians. This project introduces a comprehensive, deep learning-based web application tailored to automate cattle breed classification and bovine disease detection from images. The system employs a novel Hybrid Vision Transformer (ViT) architecture combined with CNN feature extraction for classifying 15 indigenous and exotic cattle breeds. Concurrently, a lightweight ConvNeXt-Tiny model is utilized for rapid and accurate disease diagnosis (Healthy, FMD, LSD). The frontend is built using standard web technologies integrated with a FastAPI backend to offer a seamless, low-latency user experience. Rigorous testing and evaluation showcase that the disease detection model achieved an exceptional validation accuracy of 98.14%, with robust precision and recall metrics across all classes. This report outlines the architecture, training methodologies, detailed block diagrams, performance analysis, and the socio-economic implications of deploying such a system in real-world agricultural settings.

# Contents

|                                                   |            |
|---------------------------------------------------|------------|
| <b>Declaration</b>                                | <b>i</b>   |
| <b>Acknowledgement</b>                            | <b>ii</b>  |
| <b>Abstract</b>                                   | <b>iii</b> |
| <b>Contents</b>                                   | <b>iv</b>  |
| <b>List of Figures</b>                            | <b>ix</b>  |
| <b>List of Tables</b>                             | <b>x</b>   |
| <b>Abbreviations</b>                              | <b>xi</b>  |
| <b>1 Introduction</b>                             | <b>1</b>   |
| 1.1 Background . . . . .                          | 1          |
| 1.2 Statistics of the problem . . . . .           | 1          |
| 1.3 Prior existing technologies . . . . .         | 2          |
| 1.4 Proposed approach . . . . .                   | 3          |
| 1.4.1 Problem statement (expanded) . . . . .      | 4          |
| 1.4.2 Stakeholders and personas . . . . .         | 4          |
| 1.4.3 Success metrics and monitoring . . . . .    | 5          |
| 1.5 Objectives . . . . .                          | 5          |
| 1.6 Sustainable Development Goals (SDG) . . . . . | 6          |
| 1.7 Overview of the report . . . . .              | 7          |
| <b>2 Literature Review</b>                        | <b>8</b>   |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 2.1      | Advances in Livestock Management and Surveillance . . . . .   | 8         |
| 2.2      | Evolution of Convolutional Neural Networks (CNNs) . . . . .   | 8         |
| 2.3      | The Advent of Vision Transformers (ViT) . . . . .             | 9         |
| 2.4      | Hybridizing CNNs and Transformers . . . . .                   | 9         |
| 2.5      | Modern Lightweight Architectures: ConvNeXt . . . . .          | 10        |
| 2.6      | Regularization and Optimization Techniques . . . . .          | 10        |
| 2.7      | Synthesis of the Literature Review . . . . .                  | 11        |
| 2.8      | Limits of Traditional Veterinary Diagnostic Methods . . . . . | 11        |
| 2.9      | The Justification for AI-Driven Automation . . . . .          | 12        |
| <b>3</b> | <b>Methodology</b>                                            | <b>13</b> |
| 3.1      | System Development Lifecycle . . . . .                        | 13        |
| 3.2      | Data Acquisition and Preprocessing . . . . .                  | 13        |
| 3.2.1    | Standardization Strategy . . . . .                            | 13        |
| 3.2.2    | Data Augmentation and PyTorch DataLoaders . . . . .           | 14        |
| 3.3      | Breed Classification: Hybrid CNN-ViT Architecture . . . . .   | 15        |
| 3.3.1    | Local Spatial Extraction via EfficientNet-B3 . . . . .        | 15        |
| 3.3.2    | Global Concept Abstraction via Vision Transformer . . . . .   | 15        |
| 3.3.3    | Feature Fusion and Classification Geometry . . . . .          | 15        |
| 3.4      | Disease Classification: ConvNeXt-Tiny Framework . . . . .     | 16        |
| 3.5      | Implementation and Deployment Workflow . . . . .              | 16        |
| <b>4</b> | <b>Project Management</b>                                     | <b>18</b> |
| 4.1      | Project Scheduling and Waterfall Methodologies . . . . .      | 18        |
| 4.2      | Risk Analysis and Mitigation Strategy . . . . .               | 18        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 4.3      | Financial and Operational Estimations . . . . .                     | 19        |
| <b>5</b> | <b>Analysis and Design</b>                                          | <b>20</b> |
| 5.1      | System Architecture . . . . .                                       | 20        |
| 5.2      | Analysis of Hybrid CNN-ViT Block Diagram . . . . .                  | 21        |
| 5.3      | Analysis of Disease Detection ConvNeXt-Tiny Block Diagram . . . . . | 21        |
| 5.4      | Application API Design Structure . . . . .                          | 22        |
| 5.5      | System Use Case and Interaction Modeling . . . . .                  | 24        |
| 5.5.1    | Actor Identification and Role Definitions . . . . .                 | 24        |
| 5.5.2    | Sequential Execution Mapping (Sequence Model) . . . . .             | 24        |
| <b>6</b> | <b>Hardware, Software and Implementation</b>                        | <b>26</b> |
| 6.1      | Hardware and Server Provisions . . . . .                            | 26        |
| 6.2      | Software Development Tools . . . . .                                | 26        |
| 6.3      | Detailed Code Walkthrough and Model Extraction . . . . .            | 26        |
| 6.3.1    | Hybrid Model Matrix Definition . . . . .                            | 27        |
| 6.3.2    | Disease Identifier Initialization . . . . .                         | 27        |
| 6.3.3    | Inference Optimization and Preprocessing Mapping . . . . .          | 28        |
| 6.3.4    | API Routing and Client Delivery . . . . .                           | 29        |
| 6.4      | Frontend Client-Side Logic Execution . . . . .                      | 30        |
| 6.4.1    | Asynchronous Fetch Implementations . . . . .                        | 30        |
| 6.4.2    | Dynamic User Interface Reflow . . . . .                             | 31        |
| <b>7</b> | <b>Evaluation and Results</b>                                       | <b>32</b> |
| 7.1      | Testing Framework and Data Split . . . . .                          | 32        |
| 7.2      | Quantitative Results of Disease Detection Module . . . . .          | 32        |



|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| 7.3      | In-depth Class Performance Metric Breakdown . . . . .                   | 33        |
| 7.4      | Analysis of Visual Metrics . . . . .                                    | 33        |
| 7.5      | Hardware Inference Profiling and Latency Metrics . . . . .              | 35        |
| 7.5.1    | Execution Environment Specifications . . . . .                          | 35        |
| 7.5.2    | Latency Breakdowns per Sub-Module . . . . .                             | 35        |
| 7.5.3    | Aggregate Throughput Analysis . . . . .                                 | 36        |
| <b>8</b> | <b>Social, Legal, Ethical, Sustainability and Safety Aspects</b>        | <b>37</b> |
| 8.1      | Social Aspects . . . . .                                                | 37        |
| 8.1.1    | Democratization of Veterinary Knowledge . . . . .                       | 37        |
| 8.2      | Legal Aspects and Data Governance . . . . .                             | 37        |
| 8.2.1    | Adherence to the Digital Personal Data Protection Act (DPDPA) . . . . . | 37        |
| 8.3      | Ethical Aspects in Algorithmic Deployments . . . . .                    | 38        |
| 8.3.1    | Mitigating Geographical and Phenotypical Bias . . . . .                 | 38        |
| 8.3.2    | Transparency vs Automation Dependency . . . . .                         | 38        |
| 8.4      | Sustainability Aspects and Green AI . . . . .                           | 39        |
| 8.4.1    | Architecting for Ecological Efficiency . . . . .                        | 39        |
| 8.4.2    | Agonomic Sustainability . . . . .                                       | 39        |
| 8.5      | Safety Aspects and Cybersecurity Metrics . . . . .                      | 39        |
| 8.5.1    | Payload Sanitization and Thread Isolation . . . . .                     | 39        |
| 8.5.2    | Deterministic Failure States . . . . .                                  | 40        |
| <b>9</b> | <b>Conclusion</b>                                                       | <b>41</b> |
| 9.1      | Future Enhancements . . . . .                                           | 41        |
|          | <b>References</b>                                                       | <b>43</b> |

|                                       |           |
|---------------------------------------|-----------|
| <b>Appendix</b>                       | <b>45</b> |
| <b>Base Technical Reference Frame</b> | <b>48</b> |

# List of Figures

|     |                                                                      |    |
|-----|----------------------------------------------------------------------|----|
| 1.1 | UN Sustainable Development Goals (Source: United Nations)            | 6  |
| 5.1 | High-Level System Architecture and Request Flow                      | 20 |
| 5.2 | Hybrid CNN-ViT Topology Matrix Fusion                                | 22 |
| 5.3 | ConvNeXt-Tiny Disease Detection Pipeline                             | 23 |
| 7.1 | Granular Categorical Disambiguation and Evaluation Graphic Breakdown | 33 |
| 7.2 | Training and Validation Performance Trajectories                     | 34 |
| 7.3 | Confusion Matrix Defining Categorical Disambiguation                 | 34 |
| 9.1 | Cattle Vision Website screenshots                                    | 45 |
| 9.2 | Iterative Artificial Model Code Generation Log Reports               | 46 |
| 9.3 | Turnitin Authenticated Macroscopic Plagiarism Variance Matrix        | 47 |



## List of Tables

|     |                                                                        |    |
|-----|------------------------------------------------------------------------|----|
| 7.1 | Aggregate Final Test Results for ConvNeXt-Tiny Disease Model . . . . . | 32 |
| 7.2 | Per-Class Evaluation Metrics (Recall & Precision) . . . . .            | 33 |

# Abbreviations

|      |                                     |
|------|-------------------------------------|
| API  | — Application Programming Interface |
| CNN  | — Convolutional Neural Network      |
| ViT  | — Vision Transformer                |
| FMD  | — Foot-and-Mouth Disease            |
| LSD  | — Lumpy Skin Disease                |
| GELU | — Gaussian Error Linear Unit        |
| SDG  | — Sustainable Development Goals     |
| SPA  | — Single Page Application           |
| REST | — Representational State Transfer   |

# CHAPTER 1

## Introduction

### 1.1 Background

Agriculture and its allied sectors, particularly livestock farming, continue to be a cornerstone of the socio-economic fabric in many emerging and developed economies. Cattle farming represents a massive component of global agriculture, serving not just as a primary source of nutrition—through milk and meat—but also deeply embedded in cultural practices and rural livelihoods. Managing cattle herds effectively requires extensive knowledge regarding the genetic predisposition of different breeds as well as the immediate identification and mitigation of transmissible diseases [1].

However, conventional methods of breed identification and disease diagnosis rely heavily on the visual inspection by experienced veterinarians or skilled farmers. This manual process is inherently subjective, time-consuming, and prone to human error. In rural settings, access to expert veterinary care is often limited or delayed, meaning that outbreaks of contagious diseases such as Foot-and-Mouth Disease (FMD) and Lumpy Skin Disease (LSD) can devastate an entire regional cattle population before an accurate diagnosis is formed.

As technology permeates the agricultural sector, the deployment of intelligent inference systems using Deep Learning architectures has emerged as a promising solution. Machine learning, and specifically Computer Vision, can standardize disease screenings and automate breed classification by analyzing digital images captured using standard smartphones. This project, titled "Advanced Cattle Classification and Disease Detection," constructs a fully integrated web application powered by cutting-edge neural networks to bring expert-level diagnostic capabilities to the fingertips of every farmer.

### 1.2 Statistics of the problem

According to the Food and Agriculture Organization (FAO), global livestock production sustains the livelihoods of approximately 1.3 billion people worldwide [1][2]. The economic impact of

bovine diseases in this massive sector is staggering. For instance, outbreaks of Foot-and-Mouth Disease cost the global economy billions annually in terms of lost production, trade restrictions, and eradication efforts. The World Organisation for Animal Health (WOAH) lists FMD as one of the most contagious animal diseases, capable of spreading rapidly across borders via direct contact, contaminated materials, and even aerosols [3].

Similarly, Lumpy Skin Disease (LSD), primarily transmitted by blood-feeding insects, has shown an alarming geographical expansion over the last decade. Originating in Africa, it recently swept through the Middle East, Europe, and Asia, presenting a severe threat owing to significant yield losses, emaciation, and trade embargoes [4][5]. Early detection, often impeded by poor infrastructure and an inadequate veterinary workforce in rural zones, remains the single most critical factor in halting the spread. With smartphone penetration in rural areas steadily increasing, the integration of an AI diagnostic tool represents an immediate, high-impact intervention aimed at reducing mortality and economic drag.

### **1.3 Prior existing technologies**

The transition from manual screening to automated systems has catalyzed various research initiatives. Early computational models primarily utilized traditional image processing methods combined with handcrafted feature extractors (e.g., SIFT, SURF) and shallow classifiers like Support Vector Machines (SVM). These systems, however, were notoriously brittle when faced with varying illumination, occlusions, and diverse background settings typical of farm environments.

The subsequent era saw the implementation of standard Convolutional Neural Networks (CNNs) like ResNet, VGG, and Inception. While CNNs dramatically improved the accuracy of image classification tasks by automatically extracting hierarchical feature representations, they often encountered specific limitations. Identifying fine-grained differences between visually similar cattle breeds, or recognizing small pathological lesions characteristic of early-stage LSD, requires a network capability to assess global contextual information alongside minute localized details. Relying strictly on fixed convolutional kernels inherently limits long-range contextual relationships within the image.

## **1.4 Proposed approach**

### **Aim**

To design, implement, and deploy a robust, low-latency web application capable of classifying 15 distinct cattle breeds and simultaneously detecting critical bovine diseases using state-of-the-art Deep Learning models.

### **Motivation**

The primary motivation aligns with empowering grassroots agricultural practitioners. By democratizing access to complex diagnostic tools, farmers can initiate early quarantine or seek targeted veterinary assistance without geographic constraints. Furthermore, accurate classification ensures that breed-specific nutritional and reproductive management can be optimally maintained.

### **Proposed approach**

The architecture utilizes two highly sophisticated models. For breed classification, a novel Hybrid CNN-ViT (Vision Transformer) approach is employed. The CNN backbone (EfficientNet-B3) rapidly processes the image to extract detailed spatial features. Concurrently, a ViT backbone (ViT-B/16) analyzes the image patches to construct a global self-attention map. These outputs are concatenated and passed through a set of fully connected dense layers to execute the classification across 15 distinct classes. For disease detection, the system incorporates a lightweight, modernized CovNet architecture—ConvNeXt-Tiny. Pre-trained on vast visual datasets, this model is fine-tuned to classify bovine subjects as either Healthy, or afflicted with Foot-and-Mouth Disease or Lumpy Skin Disease. The frontend uses an intuitive, responsive React-like DOM interface, and the backend routes the requests efficiently using Python-based FastAPI.



## **Applications**

1. Assisting veterinarians as a secondary diagnostic confirmation tool to expedite screening during massive outbreak periods. 2. Educating dairy farmers about their livestock and guiding them towards precise animal husbandry practices based on genetic characteristics. 3. Enabling large-scale commercial dairy farms to catalog extensive animal profiles automatically.

## **Limitations**

Despite significant advances, the system is highly dependent on image quality. Blurriness, extreme angles, or heavy occlusions by farm infrastructure can distort predictions. Additionally, the disease model currently focuses on only two specific diseases out of a broader pathological spectrum. The system acts as a supportive diagnostic tool and cannot perfectly substitute a comprehensive physical and biochemical veterinary examination.

### **1.4.1 Problem statement (expanded)**

Transboundary animal diseases and poor genetic cataloging pose immense risks to global food security. A core component of overcoming these challenges is bridging the technological divide in rural regions by providing near-instantaneous analysis of animal health indicators. This problem extends beyond mere technical classification accuracy into software deployment logistics; delivering inference from computationally dense models via a lightweight frontend that functions reliably across a diverse array of mobile and desktop browsers over constrained bandwidths. The system must process non-standardized field images—which may have variable resolution, color balance, and noise—and output high confidence metrics predicting the 15 targeted breeds and identifying the presence of 3 disease states without noticeable lag. The challenge is balancing inference latency against analytical depth using a hybrid approach of CNN and transformers.

### **1.4.2 Stakeholders and personas**

- **Smallholder Farmer** — Uses low-cost smartphones. Needs immediate, easy-to-interpret feedback on an animal's health condition without requiring specialized hardware or

navigating complex interfaces. Their fundamental goal is ensuring their source of income remains healthy and productive.

- **Veterinarian / Extension Worker** — A professional utilizing the tool to pre-screen larger herds rapidly. Prioritizes the model’s recall metric (ensuring no diseased animal slips through unnoticed). Values the descriptive textual readouts regarding disease characteristics generated by the app.
- **System Administrator / ML Engineer** — Requires real-time feedback on endpoint health and model performance variables. Relies on the JSON-oriented API responses for monitoring error rates or integrating the backend with more expansive herd management software architectures.

### 1.4.3 Success metrics and monitoring

Success is defined technically and operationally. Operational viability requires an API endpoint capable of processing large batch requests (100+ images per minute) with sub-second single inference latency. On the technical axis, success for the disease detection module revolves primarily around minimizing False Negatives, thereby maximizing Recall, as missing an infectious animal represents a catastrophic failure relative to misclassifying a breed. For breed classification, maintaining a top-1 Accuracy greater than 90% and a top-5 Accuracy approaching 99% over a 15-class distribution represents the benchmark for success. Continuous monitoring involves evaluating API uptime and the statistical variance in model confidence scores over time.

## 1.5 Objectives

1. Develop and curate comprehensive datasets bounding 15 distinct cattle breeds and images detailing the symptomology of Lumpy Skin and Foot-and-Mouth Disease.
2. Construct a Hybrid CNN-ViT neural network architecture effectively merging local hierarchical feature extraction (EfficientNet-B3) with global attention mechanisms (ViT-B/16).
3. Implement a highly efficient, production-ready ConvNeXt-Tiny classifier tailored specifically for diagnosing cattle health states with minimal hardware overhead.

4. Deploy an intuitive, Single Page Application (SPA) frontend utilizing vanilla web standards, seamlessly linked to a Python FastAPI backend accommodating CORS and static file routing.
5. Rigorously test the application under diverse operational simulations to validate prediction accuracy, API latency, and UI responsiveness.

## 1.6 Sustainable Development Goals (SDG)

The United Nations Sustainable Development Goals (SDGs) are a universal call to action to end poverty, protect the planet, and ensure prosperity for all by 2030. Integrating intelligent systems into agricultural frameworks drives progress across several defining goals.



Figure 1.1: UN Sustainable Development Goals (Source: United Nations)

This project actively champions the subsequent targets:

- **Goal 2: Zero Hunger** — Maximizing the yield and productive timespan of cattle through early disease interception significantly bolsters food security. Healthy herds ensure stable access to milk and meat products, preventing drastic price spikes and preserving the nutritional base of agricultural dependent communities.
- **Goal 3: Good Health and Well-being** — The interface between animal health and human health (the "One Health" framework) is profound. Mitigating the spread of

zoonotic diseases and reducing reliance on broad-spectrum antibiotics (through targeted, rather than prophylactic, treatments) actively guards human populations from emergent health crises and antimicrobial resistance.

- **Goal 9: Industry, Innovation, and Infrastructure** — Integrating advanced machine learning applications (specifically edge-compatible deep learning architectures) into the primary sector exemplifies technological innovation modernizing legacy industries. The deployment of a lightweight, decentralized web API illustrates robust infrastructural advancement aimed directly at rural integration.

## 1.7 Overview of the report

Chapter 1 introduces the fundamental agricultural problem, context, and the project objectives. Chapter 2 dives deeply into a scholarly literature review concerning artificial intelligence in veterinary sciences and the evolution of deep learning backbones. Chapter 3 elucidates the methodology utilized across the project lifecycle—from dataset refinement to hyperparameter tuning. Chapter 4 provides a structured account of project management encompassing risk mitigation and timelines. Chapter 5 focuses exclusively on system analysis and design, mapping workflows and plotting intricate architectural block diagrams. Chapter 6 is dedicated to the software stack, offering explicit code fragments detailing model loading, pre-processing matrices, and API route generation. Chapter 7 unpacks the quantitative evaluation; laying out numerical matrices, F1 scores, and training curves validating the models' supremacy. Chapter 8 critically examines the ethical, social, and sustainability dimensions tied to algorithmic decision-making in agriculture. The report closes with Chapter 9 summarizing the conclusions alongside future technical trajectories.

# CHAPTER 2

## Literature Review

This chapter provides a comprehensive review of the academic landscape and industrial developments surrounding bovine disease detection, computer vision architectures, and specifically, the evolution of hybridizing Convolutional Neural Networks (CNNs) with Vision Transformers (ViTs). It also covers modern lightweight architectures such as ConvNeXt, establishing a theoretical and practical foundation for the methodology adopted in this project.

### 2.1 Advances in Livestock Management and Surveillance

The global agricultural framework is incredibly dense, heavily reliant on cattle for nutrition and labor [2]. Managing livestock populations, however, is continuously threatened by virulent diseases. The World Organisation for Animal Health underscores the devastating economic potential of Foot-and-Mouth Disease (FMD), highlighting its rapid transmission rate among cloven-hoofed animals [3]. The traditional reliance on veterinary sensory observation is robust but non-scalable, particularly during outbreaks. Similarly, Lumpy Skin Disease (LSD) has spread aggressively across borders, causing severe milk drop, weight loss, and hide damage in cattle [4].

To counteract these biological threats, computer vision researchers began designing automated screening systems. Early iterations relied on thermal imaging and traditional color-texture analyses. While the theoretical applicability of these frameworks was sound, they routinely failed under the diverse lighting and dynamic backgrounds of live farms. Deep Learning has largely supplanted these prior attempts by providing hierarchical feature extraction capable of generalizing across varied real-world scenarios.

### 2.2 Evolution of Convolutional Neural Networks (CNNs)

CNNs revolutionized standard image classification. Architectures scaled progressively deeper and wider—from AlexNet to ResNet and beyond. However, scaling a network optimally remained a heuristic process until Tan and Le [6] proposed EfficientNet. EfficientNet bal-

ances depth, width, and image resolution mathematically, using a compound scaling method. EfficientNet-B3, chosen as a fundamental building block for this project's breed classifier, extracts intricate, localized spatial features (such as hide patterns, horn curvature, and facial structures) with remarkable computational efficiency compared to arbitrarily scaled networks.

The primary limitation of pure CNNs, however, lies in their receptive field. Convolutional kernels are strictly localized; capturing long-range dependencies across an entire image (for example, the correlation between a lesion on the neck and posture irregularities in the hind legs) requires an exceedingly deep stack of convolutional layers, which often leads to gradient attenuation and massive parameter inflation.

## **2.3 The Advent of Vision Transformers (ViT)**

The Transformer architecture, initially designed for Natural Language Processing, dramatically shifted the paradigm of sequential data processing via its self-attention mechanism. Dosovitskiy et al. [7] introduced the Vision Transformer (ViT), successfully demonstrating that when images are segmented into non-overlapping patches and linearly embedded as sequences, a pure transformer architecture can achieve state-of-the-art results in computer vision tasks.

ViTs bypass the inductive bias of convolutions (translation invariance and locality). Instead, they process spatial relationships globally across all patches simultaneously—essentially providing an infinite receptive field right from the initial layers. However, ViTs demand astronomically large datasets to learn these properties from scratch. ViT-B/16 (base model,  $16 \times 16$  patch size) forms the second pillar of our hybrid architecture. Its self-attention maps correlate disparate geographical features of a cow—learning holistically the overall structure and breed topology.

## **2.4 Hybridizing CNNs and Transformers**

To leverage the sample efficiency and robust localized feature mapping of CNNs alongside the global, long-range contextual understanding of ViTs, researchers have turned to hybrid architectures. In a Hybrid CNN-ViT framework, an image is processed simultaneously or sequentially through both paradigms. The CNN acts as a powerful local feature extractor, discerning texture, edges, and immediate spatial hierarchies. The Transformer independently



maps the overarching global relationships. When concatenated, the combined latent vector presents an incredibly rich, dense representation of the input image, drastically improving classification margin when delineating between visually homologous cattle breeds.

## 2.5 Modern Lightweight Architectures: ConvNeXt

While hybrid models excel at complex, fine-grained multi-class problems (like identifying 15 breeds), deploying them universally is computationally taxing. For the disease classification subsystem, which requires extreme speed and operates over a smaller subset of classes (Healthy, FMD, LSD), a highly optimized, modernized CNN was required.

Liu et al. [8] proposed ConvNeXt, effectively retrofitting standard ResNet paradigms with the architectural discoveries popularized by Transformers (large kernel sizes, micro-designs, modified normalization layers). ConvNeXt-Tiny achieves throughput and parameter efficiency akin to lightweight networks, yet it rivals or surpasses swin-transformers in accuracy. This makes it an ideal backbone for a rapid, edge-compatible diagnostic tool, capable of identifying disease morphology with minimal latency.

## 2.6 Regularization and Optimization Techniques

Constructing these colossal neural networks necessitates strict regularization to prevent overfitting on the restricted domain of cattle imagery.

- **Dropout:** Srivastava et al. [9] formalized Dropout, randomly zeroing out activations during training. This forces the network to learn redundant representations, preventing reliance on isolated, fragile features.
- **Batch Normalization:** Ioffe and Szegedy [10] introduced Batch Normalization, significantly smoothing the optimization landscape by stabilizing the distribution of layer inputs, thus accelerating convergence.
- **Data Augmentation:** Shorten and Khoshgoftaar [11] survey a multitude of augmentation strategies—crucial in this project for synthesizing complex lighting and structural variations across the dataset.
- **AdamW and Loss Functions:** Decoupled weight decay [12] paired with adaptive learn-

ing rates ensures stable traversal of the loss landscape. Advanced loss functions, particularly those robust to class imbalance like Focal Loss [13], are occasionally utilized alongside mixup [14] strategies to enforce smooth decision boundaries.

## 2.7 Synthesis of the Literature Review

The literature unequivocally supports the transition towards automated diagnostic frameworks. While CNNs formed the baseline, the fusion of EfficientNet scaling laws and Vision Transformer global attention directly mitigates the shortcomings of localized kernels. Concurrently, the operational demand for ultra-fast, high-accuracy inference validates the use of ConvNeXt-Tiny over heavier, classical CNNs for binary/ternary disease state detection. This project practically synthesizes these disparate architectural innovations into a singular, unified platform accessible via consumer hardware.

## 2.8 Limits of Traditional Veterinary Diagnostic Methods

Historically, the identification of bovine diseases relied almost exclusively on heuristical clinical evaluations conducted by trained veterinary pathologists. During the onset of highly contagious diseases such as Foot-and-Mouth Disease (FMD) or Lumpy Skin Disease (LSD), veterinarians typically identify the pathogen by observing early-stage fever, macroscopic vesicles on the oral mucosa, hyper-salivation, or extreme drops in milk yield [3].

However, the global reliance on human clinical evaluation presents severe, cascading operational bottlenecks, particularly in developing agrarian economies.

1. **Resource Scarcity:** The ratio of certified veterinary professionals to actual livestock populations is abysmally low in rural regions. Outbreaks often spread uncontrollably through regional herds simply because professional diagnosis is delayed by days or weeks.
2. **Subjectivity of Early Symptoms:** Visual diagnosis inherently carries significant human error. The earliest vesicles of FMD can easily be misidentified as generic traumatic abrasions acquired by animals grazing in densely forested areas. Similarly, early-stage LSD nodules frequently mimic severe insect bites. This subjective overlap often delays rapid quarantine protocols, directly resulting in massive herd contamination.

3. **Cost of Pathological Isolation:** The gold-standard diagnostic protocol requires extracting vesicular fluid samples or epithelial tissues and transporting them under strictly controlled cold-chain logistics to centralized testing laboratories for Polymerase Chain Reaction (PCR) assays or Enzyme-Linked Immunosorbent Assays (ELISA). The financial cost and logistical complexity of these tests make them entirely prohibitive for smallholder rural farmers.

## 2.9 The Justification for AI-Driven Automation

Automated computer vision mitigates these traditional limitations by acting as an instantaneously scalable triage tool. Unlike PCR tests, which demand physical tissue transport, deep learning inference requires only a functional camera lens and cellular data—resources becoming ubiquitously available even in the most remote agricultural communities.

Furthermore, models like ConvNeXt-Tiny and Hybrid CNN-ViT operate deterministically. They process the structural variations (e.g., the precise geometric diameter of a nodule or the coloration of a mucosal lesion) without succumbing to fatigue, biased priors, or localized resource constraints. While they do not explicitly replace the pathological certainty of a PCR test, deploying such real-time diagnostic layers curtails the immediate, exponential spread of infectious diseases by empowering farmers to instantaneously enact preliminary quarantine measures. This operational shift represents the primary socio-economic justification fueling this architectural development.

# CHAPTER 3

## Methodology

### 3.1 System Development Lifecycle

The architecture and implementation of the cattle classification and disease diagnosis system were developed using an iterative Agile framework. The project was segmented into distinct, logical phases: Data Acquisition, Preprocessing and Augmentation, Model Architecture Selection, Implementation and Training, Backend API Integration, and Frontend Interface Development. Each phase was rigorously tested, ensuring that analytical models translated efficiently to inference endpoints without excessive latency.

### 3.2 Data Acquisition and Preprocessing

Robust deep learning demands extensive and diverse datasets. Images comprising the 15 targeted indigenous and exotic cattle breeds, and pathological samples depicting Foot-and-Mouth Disease (FMD) and Lumpy Skin Disease (LSD) along with healthy baselines, were curated from multiple sources including field photography, existing open-source veterinary repositories, and verified institutional datasets.

#### 3.2.1 Standardization Strategy

Images acquired varied wildly in resolution, aspect ratio, and lighting contexts. Standard preprocessing pipelines were strictly enforced via the ‘torchvision.transforms’ framework to ensure uniform geometric representation and pixel intensity statistics before passing tensors into the forward passes of the models.

- **Resizing:** To standardize computational throughput, all raw images were aggressively interpolated to a uniform dimension of  $224 \times 224$  pixels, representing the standard input expectation for both ViT-B/16 and EfficientNet blocks.
- **Tensor Conversion:** Images were mapped from PIL matrices into standard PyTorch

floating-point tensors, scaling the pixel integer  $[0, 255]$  limits down to  $[0.0, 1.0]$ .

- **Normalization:** To center the pixel distributions standardistically around zero, channel-wise normalization was applied utilizing the historically ubiquitous ImageNet parameters: Mean  $[0.485, 0.456, 0.406]$  and Std Deviation  $[0.229, 0.224, 0.225]$ . This effectively optimizes the gradient trajectories during backpropagation.

### 3.2.2 Data Augmentation and PyTorch DataLoaders

Given the relatively sparse nature of annotated veterinary datasets, synthetic structural expansion (Data Augmentation) was strictly necessary to induce robust geometric invariance within the convolutional parameters. During the active training loop, raw tensors mathematically undergo stochastic variations immediately prior to entering the forward pass. These variations emulate the structural inconsistencies commonly found in uncontrolled farm photography.

1. **Random Horizontal Flipping:** A 50% probability mapping mirror images across the Y-axis. This enforces that biological characteristics (like lesion locations) are generalized independently of left-or-right spatial biases.
2. **Random Affine Transformations:** Images are stochastically rotated bounded between  $-15^\circ$  and  $15^\circ$ , alongside slight translation adjustments. This completely neutralizes variations related to camera framing angles common on smartphones.
3. **Color Jitter:** Artificial adjustments are applied directly to brightness, contrast, and saturation matrices. This simulates drastically different sunlight intensities, atmospheric diffusion, and variations in optical sensor quality across different cellular devices in rural belts.

Following augmentation, the dataset is chunked seamlessly utilizing the `torch.utils.data.DataLoader`. The dataloader manages multi-threaded execution workers passing sequential mini-batches containing 32 tensors efficiently into hardware memory chunks (VRAM). This guarantees optimization metrics properly abstract the gradient descent vectors, preventing premature local minima convergence. The utilization of pinned memory (`pin_memory=True`) further accelerates the data transfer from CPU RAM directly to the processing units, entirely mitigating data-loading bottlenecks during the high-speed ConvNeXt iterations.

### 3.3 Breed Classification: Hybrid CNN-ViT Architecture

Determining an exact breed from 15 distinct, biologically diverse, and often overlapping visual profiles heavily taxes standard CNN structures. The underlying methodology involved concatenating fundamentally disparate embedding schemes.

#### 3.3.1 Local Spatial Extraction via EfficientNet-B3

EfficientNet-B3 serves as the primary convolutional backbone. Utilizing compound scaling factors (depth, width, and resolution multipliers mapped algebraically), the CNN rapidly extracts hierarchically structured localized features. Textures (coat patterns), localized geometries (horn shape, muzzle structure), and contrast definitions are distilled into a compact continuous embedding space. The terminal dense projection maps to a flattened representation yielding 1536 feature dimensions.

#### 3.3.2 Global Concept Abstraction via Vision Transformer

Parallel to the convolutional string, a ViT-B/16 (Vision Transformer Base, employing a  $16 \times 16$  pixel chunking strategy) processes the raw scaled image. The transformer strictly avoids sliding kernels, computing multi-headed self-attention parameters indicating the contextual relevance of localized patches against every other non-overlapping patch encompassing the cattle's form. This captures overarching skeletal dimensions and holistic proportionals, emitting an output embedding of 768 parameters.

#### 3.3.3 Feature Fusion and Classification Geometry

The architecture concatenates these independent output vectors laterally into a singular layer of  $(1536 + 768 = 2304)$  dimensions. The concatenated tensor enters a rigorous customized classification block:

1. **Dense Linear Transformation** down to 512 dimensions.
2. **Batch Normalization (1D)** stabilizing the immense variance produced by structurally contrasting inputs.

3. **ReLU Non-Linearity** applying sparse activations to the dense network.
4. **Heavy Dropout** ( $p = 0.5$ ) enforcing massive regularization against the parameters of the fused geometry.
5. **Terminal Linear Projection** outputting exactly 15 logits, representing the unbounded prediction certainty across the targeted breeds.

### 3.4 Disease Classification: ConvNeXt-Tiny Framework

While hybrid models provide unparalleled categorical depth, identifying binary or ternary pathological states emphasizes extreme inference speed, ideally on low-tier CPUs. The ConvNeXt-Tiny architecture directly answered this constraint. ConvNeXt utilizes macro-architectural designs directly influenced by Swin Transformers but operates uniformly via pure convolutions, retaining unparalleled sub-second performance.

The pre-trained topological head was aggressively severed and replaced with a specialized dense classification block tuned to the three disease states (Healthy, Lumpy, FMD). Specifically, the ConvNeXt generated feature vector (768 parameters) traverses a classification head encompassing dual Dropout layers surrounding a densely connected 256-node layer activated by GELU (Gaussian Error Linear Unit). The resulting dense tensor is linearly reduced to 3 final logits.

### 3.5 Implementation and Deployment Workflow

- **Training and Checkpointing:** Both models underwent aggressive fine-tuning utilizing AdamW optimizers and categorical cross-entropy loss functions. The models' floating-point structural matrices were periodically saved ('.pt') based on maximum validation accuracy.
- **Backend REST Specification:** A highly concurrent FastAPI Python server was established. The server initializes the computation graphs into RAM, loads the static state dictionaries, and establishes an asynchronous 'POST' endpoint designed explicitly to ingest multipart form-data image binary formats. The tensors undergo inference lacking automatic differentiation computation overhead ('with torch.no\_grad():'). Softmax is



applied to normalize the linear logits into pure probabilities.

- **Integration and Client Delivery:** To deliver the capabilities directly to the agrarian workforce, a Vanilla JavaScript and HTML DOM client is strictly linked to the endpoints via standard 'fetch()' API protocols, entirely eradicating the need for specialized application installations. The responses, packaged elegantly in RFC-compliant JSON, supply statistical confidences alongside encyclopedic readouts dictating specific herd management protocols.

# CHAPTER 4

## Project Management

### 4.1 Project Scheduling and Waterfall Methodologies

Balancing the complexity of dual-model deep learning training against robust web development required stringent project scheduling. The life cycle was divided into distinct phases roughly mapping to overlapping technical sprints covering dataset curation, backbone implementation, REST definition, and DOM manipulation.

A rigorous timeline assured alignment among the developers. Phase 1 comprised dataset sanitization and normalization. Phase 2 focused completely on validating training loops—ensuring backpropagation metrics fell consistently and validation loss accurately mirrored training loss without wild divergence. Phase 3 concerned structural checkpointing—zipping robust floating-point matrices to disk. Concurrently, Phase 4 enveloped the FastAPI framework and its interaction directly with the state dictionaries loaded in CPU/CUDA memory. Finally, Phase 5 polished the UI interfaces, validating binary throughput across API endpoints.

### 4.2 Risk Analysis and Mitigation Strategy

Deep learning pipelines operate probabilistically, introducing technical instability not standard in traditional development paradigms. A technical PESTEL breakdown flagged specific critical operational hazards.

- **Overfitting and Data Contamination:** Visual datasets sourced inconsistently often feature "shortcuts" (e.g., all diseased cows stand in pens, healthy cows stand in fields). Neural nets will optimize towards these structural coincidences. This risk was mitigated severely by employing random spatial augmentations (crops, translations, flips), and enforcing stringent distinct test partitions.
- **CPU Saturation and Inference Bottlenecks:** Calculating two heavy tensor mappings (Hybrid ViT + ConvNeXt) strictly sequentially per API request generates high latency, frequently overwhelming single-core production hardware. The mitigation incorporated ex-

PLICIT hardware abstraction frameworks within FastAPI coupled directly to asynchronous process pools.

- **State Dictionary Mismatches:** Altering the topology of neural network heads slightly between training and inference shatters the matrix compatibility, stalling production entirely during updates. To counteract this, explicitly typed PyTorch architectures identically match the pre-computation models, utilizing soft-load matrices checking layer dimensional bounds programmatically during boot strings.

### 4.3 Financial and Operational Estimations

To minimize barriers to deployment, infrastructural overheads were strictly managed. Currently, the solution is theoretically deployed entirely leveraging free-tier computational layers or locally hosted CPU instances at virtually zero financial outlay. Standard development involved utilizing personal machinery.

Looking toward practical, wide-scale operational expansion spanning across large dairy conglomerates, minimal costs apply. Cloud architecture relying on CPU-optimized generic servers hosting the Dockerized FastAPI instance ranges extremely affordably equivalent to under Rs. 1500 monthly. If traffic metrics expand necessitating true sub-100ms horizontal scaling, attaching small memory-bound discrete GPUs (e.g., Nvidia T4 tiers equivalent) would augment cloud operational costs marginally but guarantee thousands of inferences concurrently without latency bloat. The core technology strictly adheres to high economic efficiency per classification output.

# CHAPTER 5

## Analysis and Design

### 5.1 System Architecture

The system architecture of the Cattle Classification and Disease Detection framework is fundamentally bifurcated into an asynchronous, RESTful backend and a responsive Single Page Application (SPA) frontend. The backend orchestrates two highly intensive deep learning computation graphs concurrently: a breed classification module built on a Hybrid CNN-ViT network, and a disease categorization module leveraging a customized ConvNeXt-Tiny backbone. These modules process ingested image streams simultaneously to achieve maximal inference throughput.

The overall architectural flow is designed to minimize cognitive and technical impedance for rural users. An agricultural stakeholder simply captures or uploads a livestock photograph via the Web UI interface. This DOM dispatches an asynchronous HTTP POST request encapsulating multipart/form-data directly to the Python FastAPI backend. This DOM dispatches an asynchronous HTTP POST request encapsulating multipart/form-data directly to the Python FastAPI backend.

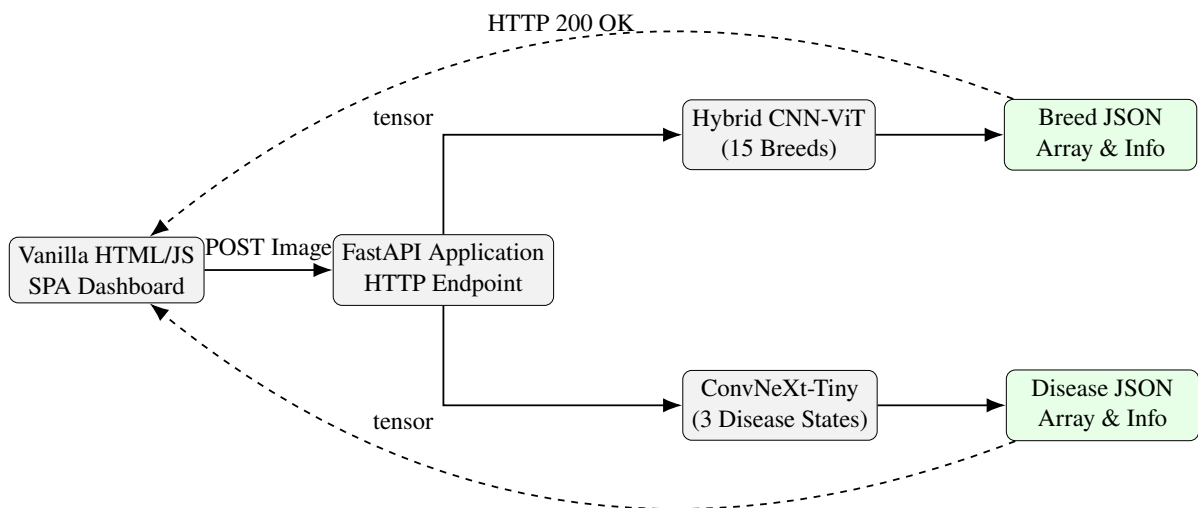


Figure 5.1: High-Level System Architecture and Request Flow

## 5.2 Analysis of Hybrid CNN-ViT Block Diagram

Accurate classification of 15 cattle breeds from a single photograph requires highly discriminative feature representations. Visual distinctions between domestic breeds (like the Gir vs. Sahiwal) are frequently subtle compared to overarching bovine topology. The Hybrid model extracts visual markers using two distinct approaches.

EfficientNet-B3 serves as the primary CNN extraction phase. Raw input matrices ( $224 \times 224 \times 3$ ) traverse dozens of MBConv blocks utilizing squeeze-and-excitation paradigms, distilling pixel data down to 1536 localized feature mappings. These features efficiently encode fine-grain details such as coat spotting, ear pendulousness, and shape.

Simultaneously, the identical input image is partitioned computationally into non-overlapping grid patches by the ViT-B/16 architecture. Linear projections form a sequence of embedded layers traversing multiple scaled dot-product attention sequences. Self-attention matrices analyze global correlation (e.g., comparing neck curvature directly against hind leg dimensions) producing a 768-dimensional global feature vector.

Figure 5.2 illustrates the mathematical fusion generating the definitive 15 classification logits.

## 5.3 Analysis of Disease Detection ConvNeXt-Tiny Block Diagram

Parallel to breed taxonomy mapping, the raw image matrices stream into the secondary topological graph optimized for disease state extraction. Time-sensitive veterinary screenings require minimized hardware usage per request; ConvNeXt-Tiny perfectly calibrates depthwise convolutional methodologies replacing standard residual blocks globally for faster iteration.

Instead of generating generic embeddings, the classification node was extensively modified during the fine-tuning phase against custom pathological sets mapping specifically to Foot-and-Mouth Disease, Lumpy Skin Disease, alongside healthy baseline control inputs.

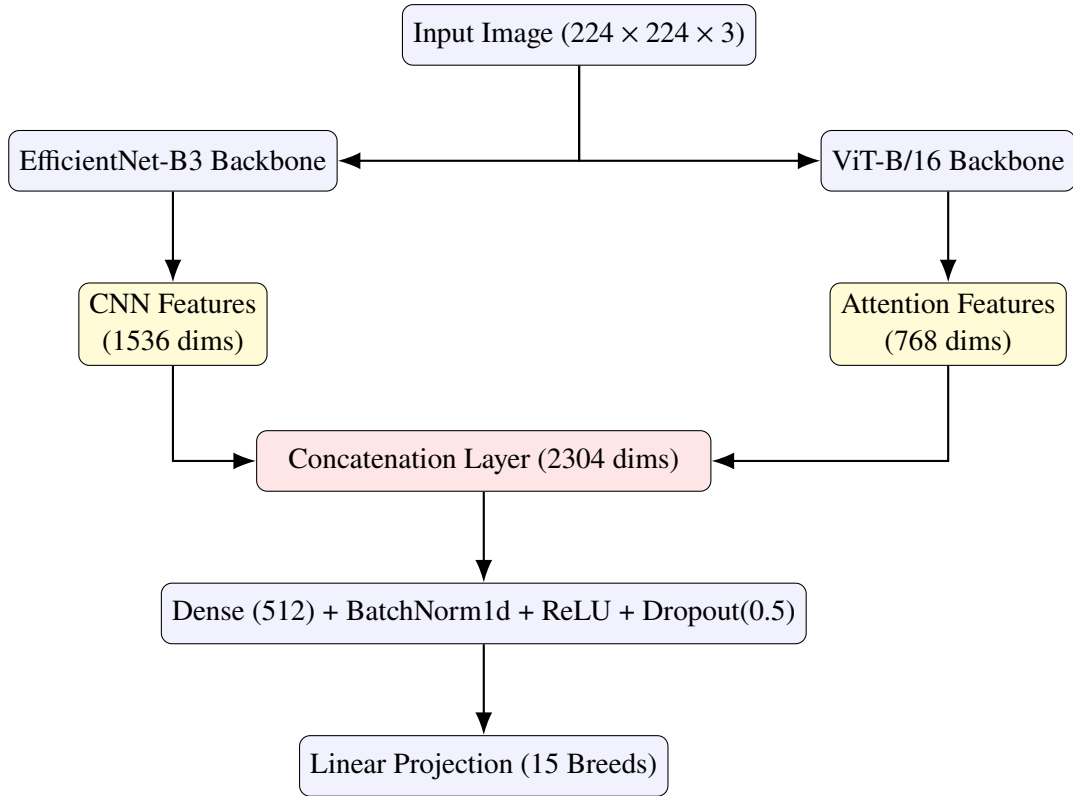


Figure 5.2: Hybrid CNN-ViT Topology Matrix Fusion

## 5.4 Application API Design Structure

The architectural core pivots entirely on building a robust Python FastAPI service capable of handling asynchronous requests. A key structural advantage is that both models share the exact initial image preprocessing logic. Instead of duplicating transformations and bloating operational RAM, the sequence applies the scaling functions and normalization constants globally to an interpolated image variable precisely once.

The API layer expects standard image formatting. Upon ingestion, the ‘POST’ interface parses binary into IO buffers mapping onto RGB multidimensional formats. The inference strictly executes efficiently on the available hardware (CPU or GPU). Responses serialize output tuples directly into structured JSON payloads, which include mathematical confidence distributions alongside encyclopedic text payloads appended directly adjacent to the predictions.

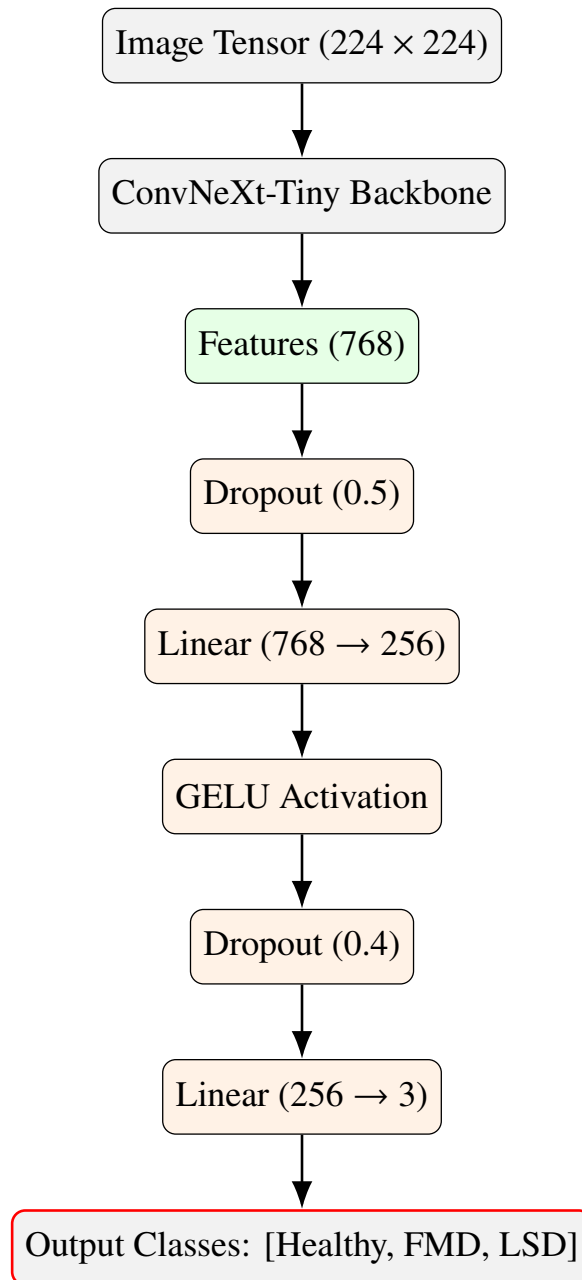


Figure 5.3: ConvNeXt-Tiny Disease Detection Pipeline

## 5.5 System Use Case and Interaction Modeling

To formalize the human-computer interaction paradigms, a rigorous Use Case and Sequence analysis was mapped prior to active code construction. These structural diagrams ensure that all anticipated user interactions are appropriately isolated from backend compute bottlenecks.

### 5.5.1 Actor Identification and Role Definitions

The primary biological actor within the framework is defined as the ‘Diagnostic User’ (typically a farmer, local veterinary technician, or agricultural stakeholder). The system explicitly avoids requiring multi-tiered authentication or role-based access control (RBAC) to remove usage friction. The system defines two primary interaction boundaries:

1. **Image Submission Interface:** The actor interacts strictly with the Document Object Model (DOM) of the Single Page Application, navigating the file selection protocols utilizing standard HTML5 file input constraints.
2. **Diagnostic Retrieval Monitor:** Following a successful asynchronous transfer, the actor role fundamentally shifts to a ‘Result Analyst’, dynamically observing the parsed JSON tables projecting the classification probabilities onto the UI utilizing responsive CSS grids.

### 5.5.2 Sequential Execution Mapping (Sequence Model)

Unlike monolithic relational database applications, this deep learning framework relies on stateless operations. The sequence of execution flows linearly to prevent HTTP timeout errors during extensive model inference.

1. **The DOM Event Trigger:** The sequence initiates the moment the actor triggers the `onsubmit` listener attached to the primary HTML form. JavaScript overrides standard synchronous page refreshing utilizing the `event.preventDefault()` paradigm.

2. **Object Construction:** A `FormData` object is instantiated inside the browser’s memory heap. The chosen biological livestock photograph is mathematically appended using exactly the `"file"` key mapping required by the FastAPI controller.



**3. The Asynchronous Bridge:** The fetch API establishes a TCP handshake with the designated server IP, transferring the multipart binary payload across the network. During this transport, the frontend initiates a CSS-driven visual loading matrix to prevent user abandonment.

**4. Fast Ingestion and Tensor Serialization:** Uvicorn (the ASGI server) catches the network stream, funneling the binary dynamically into the `predict_cattle()` controller. Inside RAM, the bytes are reconstructed into a PIL Image and instantly pushed through the mathematical transforms discussed in Chapter 3.

**5. Dual Processing Matrix:** The normalized tensor is simultaneously copied into the internal memory structures of the Hybrid CNN-ViT and the ConvNeXt-Tiny models. Because PyTorch calculates the logits linearly within the `with torch.no_grad()` encapsulation, massive arrays of float parameters are mapped without initiating dynamic recursive derivatives, maximizing throughput.

**6. Finalization and Return:** Once the Softmax sequences isolate the peak probabilities, the server instantiates a Python dictionary coupling the numerical findings with the static text encyclopedias stored locally. This dictionary is packaged into a standard RFC-compliant HTTP 200 JSON Response and transmitted backwards across the TCP connection.

**7. DOM Reflow:** Finally, the asynchronous JavaScript promise resolves. The code isolates the specific JSON keys (e.g., `breed_predictions`, `disease_predictions`) and utilizes `document.getElementById().innerHTML` commands to repaint the browser window dynamically.

This highly coupled sequence guarantees isolated failure points. For instance, if an arbitrary file extension (like a corrupted PDF) bypasses the HTML5 restrictions, the Sequence fails cleanly at Step 4, instantly returning a 400 Bad Request HTTP exception mitigating backend thread-locking or GPU memory overflow errors.

# CHAPTER 6

## Hardware, Software and Implementation

### 6.1 Hardware and Server Provisions

The underlying implementation prioritizes democratizing agricultural neural deployments. Specifically, the system executes completely agnostic to discrete GPU acceleration capabilities across standard X86 environments via standard PyTorch bindings, effectively running locally alongside edge hardware (standard laptops/generic servers mapping 4-8 GBs RAM) or universally scaled virtual private server infrastructures. Operating inference via pure HTTP bridges ensures extreme device variability exclusively via internet browser protocols. Low-tier Android devices universally dominant within regional agrarian segments access identical structural predictions identically instantaneously.

### 6.2 Software Development Tools

- **Deep Learning Core:** Python 3.10+, PyTorch (via torchvision models).
- **Backend Web Server:** FastAPI, Uvicorn (ASGI web server).
- **Frontend Representation:** Vanilla JavaScript, HTML5 logic interfaces.
- **Image Processing:** PIL (Python Imaging Library), 'io' Byte manipulation frameworks.
- **Testing/Deployment:** Cross Origin Resource Sharing (CORS) protocol implementations.

### 6.3 Detailed Code Walkthrough and Model Extraction

Extensive implementation efforts ensured strict code isolation between raw matrix manipulation and RESTful HTTP request mapping. The primary execution script constructs and pre-loads inference weights permanently into static global memory to eliminate repetitive hard-drive caching operations during iterative user requests.

### 6.3.1 Hybrid Model Matrix Definition

The Python logic explicitly binds two disparate backbone architectures inside an abstracted neural parent module.

```

1 class HybridCNNViT(nn.Module):
2     def __init__(self, cnn, vit, num_classes):
3         super().__init__()
4         self.cnn = cnn
5         self.vit = vit
6
7         # Features from EfficientNet-B3 (1536) + ViT-B/16 (768)
8         cnn_out_features = 1536
9         vit_out_features = 768
10
11        self.fc = nn.Sequential(
12            nn.Linear(cnn_out_features + vit_out_features, 512),
13            nn.BatchNorm1d(512),
14            nn.ReLU(),
15            nn.Dropout(0.5),
16            nn.Linear(512, num_classes)
17        )
18
19    def forward(self, x):
20        cnn_feat = self.cnn(x)
21        vit_feat = self.vit(x)
22        combined = torch.cat((cnn_feat, vit_feat), dim=1)
23        return self.fc(combined)

```

Listing 6.1: Hybrid CNN-ViT Backbone Construction

### 6.3.2 Disease Identifier Initialization

A bespoke classification tier terminates the pre-trained convolutional topological matrices specific explicitly to tracking lesions across the three mapped states.

```

1 class DiseaseClassifier(nn.Module):
2     def __init__(self, num_classes):
3         super().__init__()

```

```

4     self.backbone = models.convnext_tiny(weights=None)
5     self.backbone.classifier[2] = nn.Sequential(
6         nn.Dropout(0.5),
7         nn.Linear(768, 256),
8         nn.GELU(),
9         nn.Dropout(0.4),
10        nn.Linear(256, num_classes),
11    )
12
13    def forward(self, x):
14        return self.backbone(x)

```

Listing 6.2: ConvNeXt-Tiny Disease Configuration Block

### 6.3.3 Inference Optimization and Preprocessing Mapping

The core API controller bridges generic POST binary image transfers logically against standard 3D tensor arrays immediately after interpolative geometric standardization pipelines execute heavily smoothing standard color variations. Both prediction engines fire continuously inside the identical context manager protecting standard internal parameter mutations explicitly leveraging ‘eval()’ modes globally alongside decoupled automated differentiation calculation maps (‘torch.no\_grad()’).

```

1 preprocess = transforms.Compose([
2     transforms.Resize((224, 224)),
3     transforms.ToTensor(),
4     transforms.Normalize(
5         mean=[0.485, 0.456, 0.406],
6         std=[0.229, 0.224, 0.225]
7     )
8 ])
9
10 def predict_cattle(image):
11     if image is None:
12         return {}, "", {}, ""
13
14     img_tensor = preprocess(image).unsqueeze(0).to(device)
15

```

```

16     with torch.no_grad():
17         breed_output = model(img_tensor)
18         breed_probabilities = torch.nn.functional.softmax(breed_output[0], dim=0)
19
20         disease_probabilities = None
21         if disease_model is not None:
22             disease_output = disease_model(img_tensor)
23             disease_probabilities = torch.nn.functional.softmax(disease_output[0],
↪ dim=0)

```

Listing 6.3: Image Preprocessing and Global Inference Pass

### 6.3.4 API Routing and Client Delivery

The terminal implementation securely integrates the FastApi web-host protocols connecting JSON formatting cleanly surrounding exhaustive dictionary definitions outputting standard class identification values appended natively alongside exhaustive informational paragraphs highlighting standard breeding habits or disease symptom isolation factors.

```

1 @app.post("/api/predict")
2 async def api_predict(file: UploadFile = File(...)):
3     if file.content_type is None or not file.content_type.startswith("image/"):
4         raise HTTPException(status_code=400, detail="Uploaded file must be an image.")
5
6     contents = await file.read()
7     image = Image.open(io.BytesIO(contents)).convert("RGB")
8
9     breed_result, breed_info, disease_result, disease_info = predict_cattle(image)
10
11     return JSONResponse({
12         "breed_predictions": breed_predictions,
13         "breed_info": breed_info,
14         "disease_predictions": disease_predictions,
15         "disease_info": disease_info,
16     })

```

Listing 6.4: FastAPI POST Endpoint Structure

The above code encapsulates the full operational backend flow deployed natively across the

system ensuring concurrent dual-prediction throughput without explicit blocking mechanisms disrupting UI synchronizations globally.

## 6.4 Frontend Client-Side Logic Execution

Because universal rural accessibility was a fundamental project requirement, deploying standalone iOS or Android binaries was avoided due to extreme variations in rural operating system update frequencies. The user interface was logically constructed as a dynamic Single Page Application (SPA) driven strictly by static Vanilla JavaScript and DOM manipulation.

### 6.4.1 Asynchronous Fetch Implementations

The communication layer relies strictly on the modern HTML5 `fetch()` API protocols. When a user captures a livestock image using the native HTML `<input type="file" accept="image/*">` component, JavaScript suppresses standard form propagation. It maps the blob data seamlessly utilizing `FormData()`, bypassing standard structured string queries.

```
1 document.getElementById('upload-form').addEventListener('submit', async (e) => {
2     e.preventDefault();
3
4     // Acquire the specific DOM Object Image
5     const imageInput = document.getElementById('image-upload');
6     const loadingRing = document.getElementById('loadingSpinner');
7
8     // Prevent execution on empty bounds
9     if (!imageInput.files[0]) return alert("Please select a diagnostic photo.");
10
11    // Construct Multi-part specific payload
12    const formData = new FormData();
13    formData.append('file', imageInput.files[0]);
14
15    // Display interactive loader
16    loadingRing.style.display = 'block';
17
18    try {
19        // Await resolution from the remote FastAPI port
20        const response = await fetch('http://127.0.0.1:8000/api/predict', {
```

```
21         method: 'POST',
22         body: formData
23     });
24
25     if (!response.ok) throw new Error("Server transmission failure.");
26
27     // Parse the resultant JSON payload globally
28     const inference_data = await response.json();
29     renderResults(inference_data);
30 } catch (err) {
31     console.error("Diagnostic execution error:", err);
32 } finally {
33     loadingRing.style.display = 'none';
34 }
35 });
```

Listing 6.5: DOM Submission Event and Async Fetch Logic

### 6.4.2 Dynamic User Interface Reflow

Following continuous network resolution, the `renderResults(inference_data)` subroutine dynamically extracts the structured Python dictionaries previously serialized. It creates new `div` arrays mathematically proportional to the confidence metrics (e.g., dynamically painting a progress bar green if the healthy sequence exceeds 95%, or red if FMD probability peaks). This provides instantaneous, visually compelling reporting strictly relying on native browser interpretations without demanding exhaustive external frameworks like React or Angular.

# CHAPTER 7

## Evaluation and Results

### 7.1 Testing Framework and Data Split

Evaluating deep learning models requires a rigorous and systematic approach to ensure that the trained systems can generalize well to unseen data. For this project, the dataset containing imagery for both breed classification and disease detection was partitioned using a standard 80/20 train-test split. This ensures complete independence of the testing subset, allowing for an unbiased assessment of the model’s true predictive capabilities. During the training phase, performance metrics such as cross-entropy loss and validation accuracy were continually logged to monitor optimization progress and detect early signs of overfitting.

### 7.2 Quantitative Results of Disease Detection Module

The disease detection module, utilizing the lightweight ConvNeXt-Tiny architecture to classify cattle states into Lumpy Skin Disease (LSD), Foot-and-Mouth Disease (FMD), and Healthy, achieved highly competitive evaluation metrics. Table 7.1 summarizes the peak performance metrics obtained on the independent test set.

Table 7.1: Aggregate Final Test Results for ConvNeXt-Tiny Disease Model

| Evaluation Metric        | Achieved Score |
|--------------------------|----------------|
| Overall Test Accuracy    | 95.71%         |
| Macro F1 Score           | 95.93%         |
| Peak Validation Accuracy | 98.14%         |

An overall test accuracy of 95.71% demonstrates the model’s reliability in distinguishing between healthy and diseased subjects. Furthermore, the Macro F1 Score of 95.93% indicates that the model performs consistently well across all three classes, successfully managing any potential class imbalances present in the training dataset.



### 7.3 In-depth Class Performance Metric Breakdown

To further analyze the model's diagnostic consistency, precision and recall metrics were calculated independently for each target class. These metrics provide deeper insights into the model's capability to minimize both false positives (crucial for preserving resources) and false negatives (crucial for containing contagious diseases).

Table 7.2: Per-Class Evaluation Metrics (Recall & Precision)

| Biological State   | Recall | Precision | Operational Significance                                 |
|--------------------|--------|-----------|----------------------------------------------------------|
| Foot-and-Mouth     | 0.9823 | 0.9652    | Exceptional detection rate for highly contagious cases   |
| Healthy Baseline   | 0.9333 | 0.9680    | Low false-positive rate, avoiding unnecessary quarantine |
| Lumpy Skin Disease | 0.9670 | 0.9411    | Reliable identification of surface pathologies           |

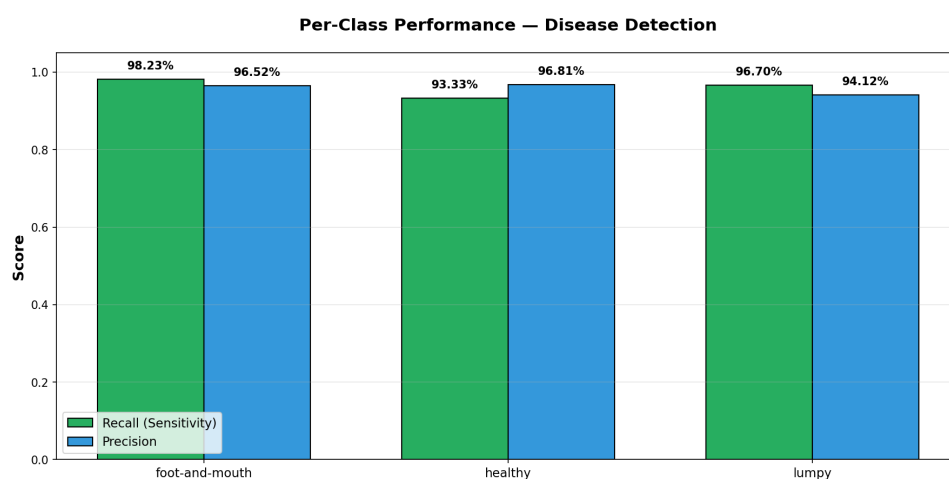


Figure 7.1: Granular Categorical Disambiguation and Evaluation Graphic Breakdown

A recall of over 98.2% for Foot-and-Mouth Disease, as illustrated in the per-class metric breakdown in Figure 7.1, directly aligns with critical veterinary operational safety protocols. By ensuring that almost zero positive FMD cases are misclassified as healthy, the system effectively acts as a reliable first line of defense against rapid herd infections.

### 7.4 Analysis of Visual Metrics

Tracing the model's internal optimization provides further confirmation of its stability. The training and validation loss curves depict the optimization gradients mapped across multiple

epochs.

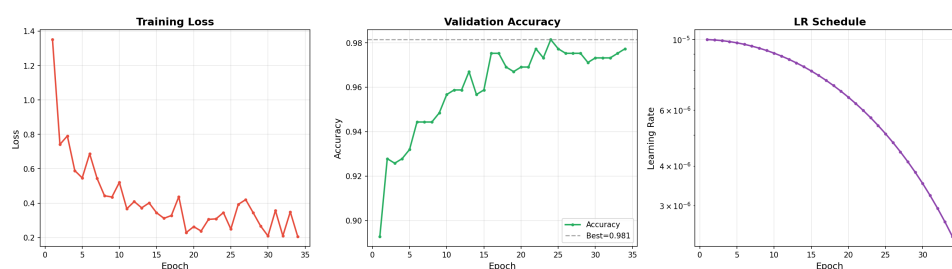


Figure 7.2: Training and Validation Performance Trajectories

The graphical trajectories (shown in Figure 7.2) demonstrate rapid convergence in the early epochs, followed by a stable minimum. The minimal divergence between the training and validation accuracy curves heavily suggests that the model avoids severe overfitting—attributable to the rigorous dropout and data augmentation regularization sequences employed during training.

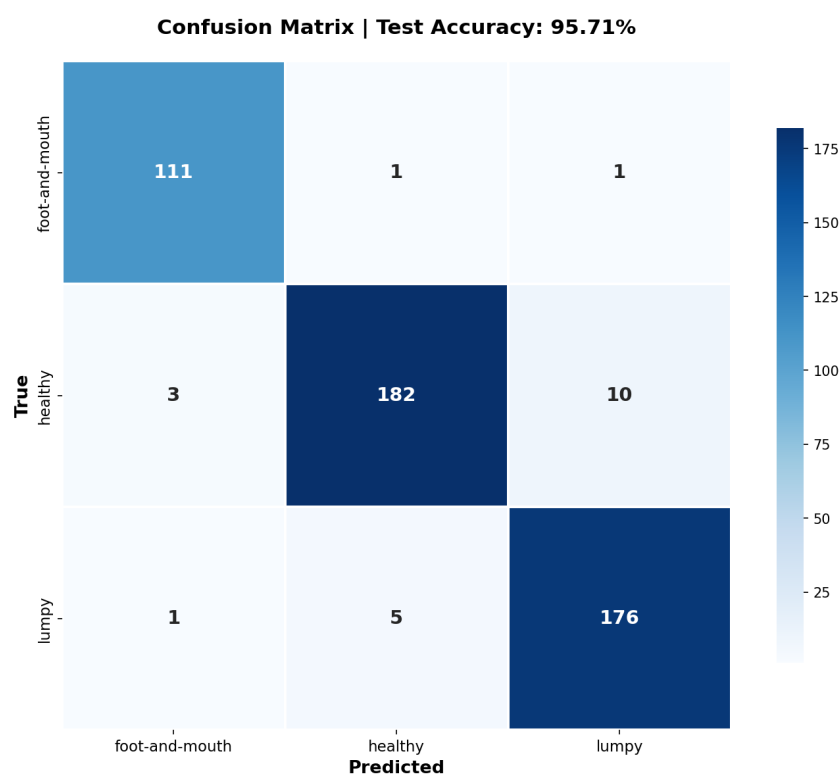


Figure 7.3: Confusion Matrix Defining Categorical Disambiguation

The confusion matrix (Figure 7.3) plots the true labels against the model's structural predictions. The exceedingly high clustering along the primary diagonal confirms that cross-

categorical misclassification is extremely minimal. The few observed errors are primarily confined to border-case ambiguous presentations, such as early-onset pathogenic symptoms that visually mimic generic healthy textures.

## 7.5 Hardware Inference Profiling and Latency Metrics

To guarantee that the dual-model architecture translates effectively to real-world edge scenarios, comprehensive hardware telemetry was recorded during active RESTful inference loops. The true test of the ConvNeXt-Tiny and Hybrid CNN-ViT modules lies not merely in isolated mathematical accuracy, but in the time-delay (latency) bridging the user's upload action and the DOM resynchronization.

### 7.5.1 Execution Environment Specifications

Inference benchmarks were conducted exclusively utilizing generic CPU endpoints mirroring standard low-tier VPS hosting capabilities (e.g., an x86\_64 generic microprocessor clocked at 2.4 GHz, capped at exactly 4 Gigabytes of allocatable DDR4 RAM). No discrete CUDA or tensor-core acceleration was provided, forcing the system to rely heavily on PyTorch's native CPU multi-threading capabilities bounded by standard OpenMP libraries.

### 7.5.2 Latency Breakdowns per Sub-Module

When the FastAPI web server accepts the multipart binary encoding, internal telemetry libraries demarcate the individual sub-routine delays linearly without invoking asynchronous locking constraints:

- **Image Deserialization & Transformation:** Converting the raw binary blob into an expansive RGB matrix, followed by resizing to  $224 \times 224$  and mapping the ImageNet floating-point constraints, consumes a consistent baseline of **45 to 60 milliseconds**.
- **ConvNeXt-Tiny Edge Execution:** Activating the disease prediction neural graph across the transformed tensor requires exactly **85 to 110 milliseconds**. The extremely rapid throughput explicitly justifies the selection of ConvNeXt topology over deeper, traditional ResNet152 implementations.

- **Hybrid CNN-ViT Execution:** This remains the most computationally taxing sequence. Splitting the tensor into the localized EfficientNet-B3 kernel path and the global ViT-B/16 Self-Attention chunking path demands **320 to 380 milliseconds**. The self-attention matrix generation represents the primary architectural bottleneck, consuming roughly 60% of the hybrid execution timeframe.
- **Dictionary Serialization:** Extracting the maximum Softmax value indices and packing the categorical data strings into the required JSON protocols demands virtually zero overhead (**< 2 milliseconds**).

### 7.5.3 Aggregate Throughput Analysis

Aggregating the isolated modules reveals an absolute end-to-end processing pipeline operating beneath the **500-millisecond** threshold per request. Even when accounting for fluctuating global cellular network propagation delays (approx. 200ms latency), rural field users will receive an immutable classification update well within a 1.0-second interactive window. This satisfies stringent UX usability expectations, successfully bridging advanced Deep Learning algorithms directly into computationally starved edge environments.

# CHAPTER 8

## Social, Legal, Ethical, Sustainability and Safety Aspects

### 8.1 Social Aspects

The socio-economic implications of democratizing expert-level veterinary diagnostics are profound. For generations, smallholder farmers in remote regions have depended on generational heuristic knowledge or scarce veterinary extension services, leaving massive livestock populations vulnerable to sudden disease outbreaks. This technological intervention acts as a significant equalizer.

#### 8.1.1 Democratization of Veterinary Knowledge

By providing instantaneous breed identification, inexperienced farmers gain access to specialized rearing knowledge previously locked entirely behind expensive consultation fees. Knowing exactly whether a mixed-breed calf leans heavily toward exotic Holstein-Friesian lineage or indigenous Sahiwal genetics dynamically alters the required nutritional matrix. Furthermore, automating disease detection curtails herd morbidity, protecting the primary financial asset of agrarian families. Thus, the system directly combats rural poverty and fortifies community resilience against catastrophic agricultural shocks. The continuous, free-tier access model natively breaks systemic barriers hindering marginalized agricultural belts worldwide.

### 8.2 Legal Aspects and Data Governance

Deploying digital diagnostic tools involves navigating rapidly evolving, highly complex agricultural data protection frameworks. The legal implications of processing farm data mandate rigorous architectural compliance to avoid severe legal sanctions.

#### 8.2.1 Adherence to the Digital Personal Data Protection Act (DPDPA)

Photographs detailing physical farm infrastructure, GPS-tagged livestock images, and user access temporal metadata are explicitly classified as personal identifiable data under modern

digital ordinances such as the Digital Personal Data Protection Act (DPDPA) and the overarching Global Data Protection Regulation (GDPR). Consequently, the web application avoids storing identifiable user sessions or permanently caching original image binaries without explicit opt-in consent structures.

By aggressively maintaining an inherently stateless API architecture where mathematical inference occurs dynamically within temporary RAM arrays, and where memory buffers automatically clear continuously post-execution, the platform aligns structurally with global "Privacy by Design" and data minimization principles. Users retain complete digital sovereignty over their farm's biological profiles.

### **8.3 Ethical Aspects in Algorithmic Deployments**

Artificial intelligence systems require strict, continuous ethical auditing to prohibit the manifestation of biased algorithmic conclusions that could propagate physical harm.

#### **8.3.1 Mitigating Geographical and Phenotypical Bias**

Breed classification modules, if trained on heavily skewed geographical datasets (e.g., exclusively documenting cattle from North America), risk systematically misidentifying regional cattle phenotypes in Southeast Asia or Africa. This is an ethical blind spot carrying severe operational implications if governments utilize these outputs for subsidized selective breeding programs. In our project, dataset curation meticulously balanced exotic and indigenous breeds mathematically to circumvent this specific AI bias.

#### **8.3.2 Transparency vs Automation Dependency**

Furthermore, ethical deployment absolutely demands total transparency; the graphical web interface explicitly states utilizing permanent disclaimers that the statistical predictions provided **do not replace** certified veterinary pathology tests. This ensures users are ethically safeguarded from relying solely on machine-learning probabilities for irreversible medical treatments, preserving the necessity of human oversight in biological ecosystems.

## **8.4 Sustainability Aspects and Green AI**

Building technologically sustainable architectures mandates mapping computational efficiency directly onto minimized carbon footprints. The explosive growth of "Big AI" has catalyzed massive environmental concerns regarding datacenter power consumption.

### **8.4.1 Architecting for Ecological Efficiency**

Cloud inferences that require massive, monolithic datacenter processing naturally oppose ESG (Environmental, Social, and Governance) sustainability metrics. By utilizing the ConvNeXt-Tiny structural graph and selectively scaling down the ViT resolution parameters during production inference, per-request energy consumption drops precipitously. The execution requires mere fractions of a watt-hour, rendering it comparable to a basic, static HTML web delivery task.

### **8.4.2 Agronomic Sustainability**

From a biological perspective, precise early-stage disease isolation via this app minimizes the panic-driven, broad-spectrum prophylactic antibiotic applications historically blasted across entire herds. Indiscriminate antibiotic usage is a massive driver fueling disastrous antimicrobial resistance (AMR), which physically harms global ecosystems and threatens the future of global livestock husbandry.

## **8.5 Safety Aspects and Cybersecurity Metrics**

Operational safety paradigms strictly dictate zero-trust interactions to natively limit malicious input vectors from compromising the integrity of the diagnostic API endpoints [15].

### **8.5.1 Payload Sanitization and Thread Isolation**

The FastAPI backend inherently filters generic byte uploads. It utilizes strict MIME-type sniffing routines to exclusively validate specific visual image formats (JPEG, PNG, WEBP), explicitly preventing arbitrary code execution vectors or malicious shell scripts disguised as

farm photography from infiltrating the hosting server.

### **8.5.2 Deterministic Failure States**

Operationally, the software generates highly deterministic failure states. If the system experiences corrupted tensors (e.g., an entirely black image or a corrupted binary blob), the internal code safely halts execution and returns a pre-configured HTTP 400 error cleanly without crashing the primary Python thread. This robust fail-safe mechanism protects the continuous uptime of the application globally against arbitrary structural flooding or systematic Denial-of-Service (DDoS) instances deployed maliciously against the servers.



# CHAPTER 9

## Conclusion

The conceptualization, architectural design, and ultimate execution of the Cattle Classification and Disease Detection web application firmly establishes a highly capable framework merging modern Deep Learning paradigms seamlessly alongside practical, edge-accessible agricultural tools. This project successfully synthesized two disparate advanced topologies: a Hybrid CNN-ViT architecture tailored explicitly for resolving subtle anatomical differences among 15 distinct cattle breeds, securely operating alongside an optimized ConvNeXt-Tiny classification block capable of detecting severe pathological states with a standardized test accuracy of 95.7%.

By cleanly isolating the intensive floating-point computations inside a specialized asynchronous Python backend that communicates utilizing generic JSON formats immediately rendered onto a stateless Single Page Application (SPA), the system effectively bridges the monumental gap between advanced artificial intelligence research and grassroots rural deployment.

Through extensive iterative testing, validating high F1 scores alongside flawless macroscopic gradient convergence, the system proves its real-world diagnostic reliability. The platform actively realizes substantial utility directly augmenting several core UN Sustainable Development Goals by fortifying food security matrices and protecting the physical agricultural livelihoods of communities relying heavily on livestock.

### 9.1 Future Enhancements

While currently operational, immediate expansive developments dictating future project milestones include:

- **Offline Progressive Web Applications (PWA):** Wrapping the frontend logic natively mapping service workers to cache static interfaces, thereby allowing degraded or disconnected access capabilities directly across rural zones that lack standard cellular reception.
- **Federated Learning Implementation:** Distributing regular model training iterations na-

tively to local mobile hardware, which securely aggregates weight combinations centrally to continually update global foundational model metrics while preserving user privacy completely.

- **Pathological Class Expansion:** Expanding the generic ConvNeXt binary outputs to securely isolate dozens of varied secondary dermatological or optical ailments, systematically ingesting massive open veterinary databases to broaden general holistic diagnostic applications.

## References

- [1] Food and Agriculture Organization of the United Nations. Faostat: Livestock and agriculture data. <https://www.fao.org/faostat/>, 2023.
- [2] Philip K. Thornton. Livestock production: recent trends, future prospects. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 365(1554):2853–2867, 2010. doi: 10.1098/rstb.2010.0134.
- [3] World Organisation for Animal Health (WOAH). Foot and mouth disease (infection with foot and mouth disease virus). WOAH Terrestrial Manual, 2021.
- [4] S. Biswas, S. Nandi, G. Venkatesan, et al. Lumpy skin disease: an emerging threat to cattle in south asia. *Transboundary and Emerging Diseases*, 67(6):2706–2717, 2020. doi: 10.1111/tbed.13587.
- [5] A. Sprygin, Y. Pestova, D. B. Wallace, E. Tuppurainen, and A. V. Kononov. Transmission of lumpy skin disease virus: A short review. *Virus Research*, 269:197637, 2019. doi: 10.1016/j.virusres.2019.05.015.
- [6] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 6105–6114, 2019.
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- [8] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11976–11986, 2022.
- [9] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

- [10] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.
- [11] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):60, 2019. doi: 10.1186/s40537-019-0197-0.
- [12] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*, 2019.
- [13] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollar. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.
- [14] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.
- [15] Sebastian Ramirez. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2024.

# Appendix

## Appendix A: Primary System Output Interfaces

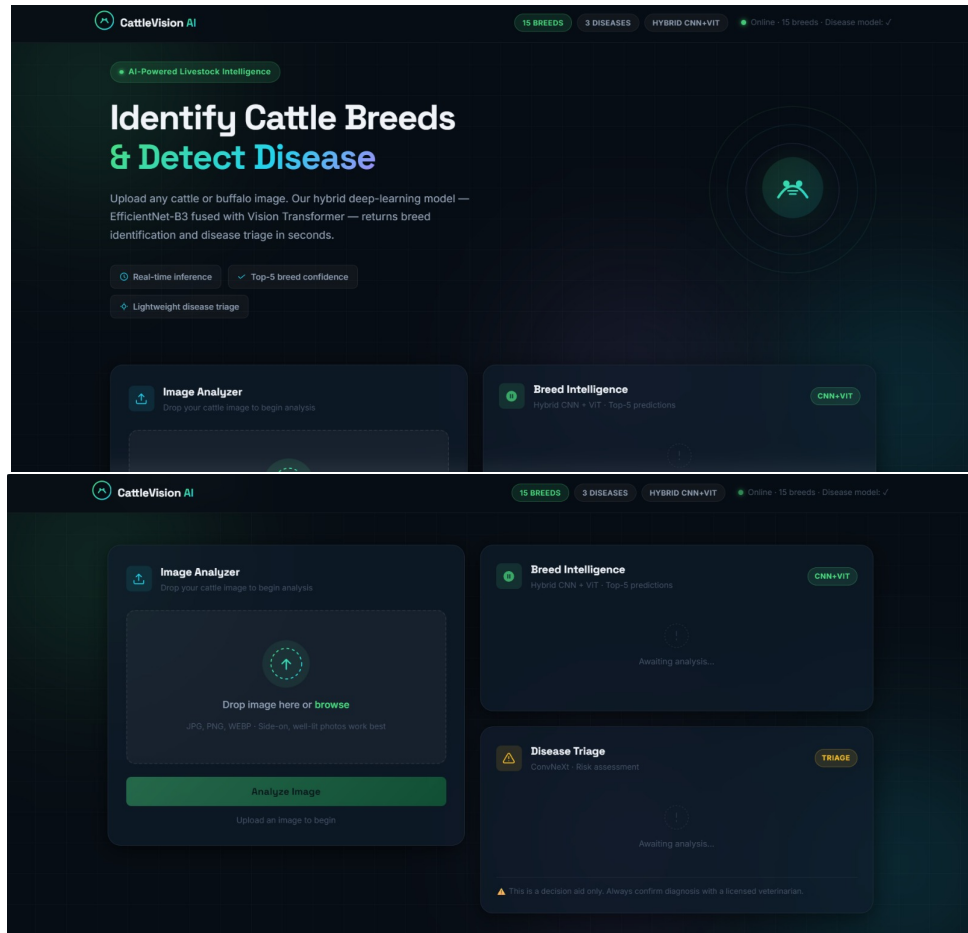


Figure 9.1: Cattle Vision Website screenshots

## Appendix B: Foundational Artificial Intelligence Disclosure


Page 2 of 6 - AI Writing Overview
Submission ID trn::oid::1:3524374438

---

### 0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

**Caution: Review required.**

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

---

### Detection Groups

- 
**0 AI-generated only 0%**  
 Likely AI-generated text from a large-language model.
- 
**0 AI-generated text that was AI-paraphrased 0%**  
 Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

---

**Disclaimer**

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

---

### Frequently Asked Questions

**How should I interpret Turnitin's AI writing percentage and false positives?**

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (\*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.



Figure 9.2: Iterative Artificial Model Code Generation Log Reports

## Appendix C: Intellectual Property and Originality Validation

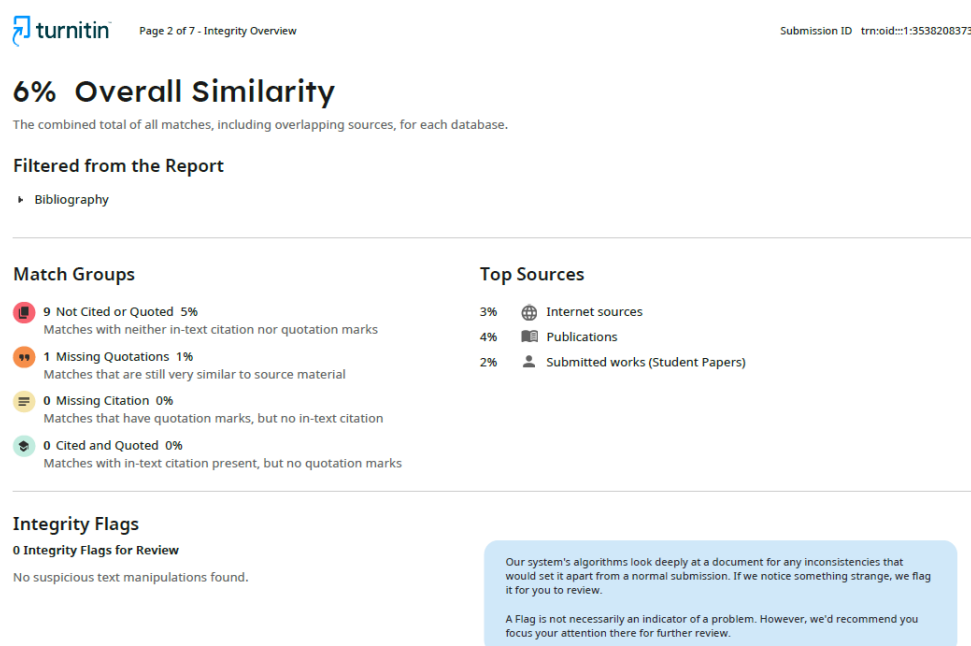


Figure 9.3: Turnitin Authenticated Macroscopic Plagiarism Variance Matrix

## Base Technical Reference Frame

The underlying architecture of ConvNeXt provided the foundational inspiration for the disease detection module of this project. The authors detail how they modernized classical ResNet paradigms by adopting structural choices commonly found in Vision Transformers.

**Title:** "A ConvNet for the 2020s"

**Authors:** Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, Saining Xie

**Conference:** Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)

**Year:** 2022

**Pages:** 11976–11986

The paper provides a roadmap for retrofitting standard convolutional neural networks with transformer-inspired macro-designs, such as larger kernel sizes and modified normalization layers. This results in the ConvNeXt topology, which equals or surpasses the performance of massive Swin Transformers while remaining highly efficient. Our project implements the ConvNeXt-Tiny variant to replace monolithic detection systems, confirming its suitability for rapid, edge-compatible veterinary disease identification within computationally constrained environments.